# ASignX system state and development review

### Build review | 19 October 2026 | Local working tree

ASignX is a working landmark-based recognition prototype with static alphabet recognition, a two-class dynamic model, collection/import tools and an early browser SDK. It is not yet a validated general sign-language translator. The next useful work is to establish reliable data and live evaluation before expanding the neural networks or vocabulary.

The most important result is the canonical static model's validation accuracy of 97.64% versus test accuracy of 53.19%. The saved dynamic model reports 100%, but its test set contains only two clips. These results do not support a claim of reliable recognition for new users.

This review inspects actual source, checkpoints, split manifests and the current CSV. It preserves the older docs/ai-state-review.md as historical evidence; that report's 26-label dataset and missing-dynamic-model statements no longer describe this build. No production model was retrained or altered for this review.

| Component | Current evidence | Assessment |
|---|---|---|
| Hand tracking | MediaPipe Tasks; up to two hands, 21 points each | Implemented; camera accuracy/latency not benchmarked here |
| Static recognition | Canonical 24-label residual MLP | Usable baseline; serious held-out generalization problem |
| Dynamic recognition | J/Z checkpoint, TCN plus attention | Experimental; only 11 clips in recorded split |
| Data tools | Static importer, CSV merger, manual and dynamic studios | Substantial collection infrastructure already present |
| Browser | TypeScript helpers and export code | Integration foundation; no verified deployed browser pipeline |

## Reading guide

Pages cover recognition, training, measured evaluation, bottlenecks, a prioritized roadmap and the update procedure. Facts are distinguished from suspected causes and proposed acceptance criteria. Repository paths are relative to the project root.

# 1. Recognition models and methods

The pipeline is camera/image -> MediaPipe hand landmarks -> normalization -> learned sign classifier -> confidence threshold -> temporal stabilization -> displayed/emitted sign. Detection and sign recognition are separate models: Google's bundled hand_landmarker.task estimates geometry; ASignX's PyTorch checkpoints learn the label vocabulary.

### Static model

Each hand supplies 21 x/y/z landmarks, flattened into 63 floats. The wrist is translated to zero and coordinates are divided by the maximum distance from that wrist. The canonical checkpoint then reflects x for detector-labeled Left hands using left_x_reflection_v1. This reduces left/right variation; it does not remove all camera viewpoint differences or guarantee linguistic equivalence of mirrored signs.

The network enriches 63 values into 91: original coordinates, 15 joint-angle cosines, 10 fingertip distances and a 3D palm normal. Dense layers of width 128 and 64, LayerNorm, GELU, dropout and residual blocks learn a compact embedding. A cosine prototype head compares that embedding with learned class vectors using a learned scale. This is a kinematic residual multilayer perceptron, not an image CNN.

The canonical checkpoint has 63,681 parameters and 24 labels: A-I and K-Y. J and Z are intentionally absent from this static vocabulary. The older sign_classifier.pth contains 26 labels and 63,809 parameters; it uses legacy preprocessing and a different dataset hash. Treat it as a separate historical model, not an interchangeable default.

### Dynamic model

The actual sequence checkpoint uses backbone=tcn_attention, although its architecture metadata says lstm for wrapper compatibility. It accepts 30 timesteps x 126 values: stable Left and Right slots, each containing 63 coordinates. Kinematic enrichment produces 182 values per timestep. A 128-wide projection feeds two residual temporal-convolution blocks with dilations 1 and 2, four-head self-attention, learned temporal pooling, a 64-wide embedding and a cosine head. It has 296,641 parameters and predicts only J or Z.

The code also offers an LSTM backbone with mean/last pooling and optional bidirectionality. That option is implemented but is not the backbone of this saved dynamic checkpoint. Two-hand input support does not establish accuracy on two-hand signs; J/Z are the only trained dynamic labels found.

Sequence normalization uses one clip-wide wrist origin and median hand scale so wrist travel and relative hand positions survive. Missing detections are interpolated, and complete clips are resampled to 30 steps. The model receives hand geometry, not facial expression, body pose, language grammar or sentence context. A body-reference helper exists, but it is not evidence of a trained body-aware recognition pipeline.

Evidence: asignx/detector.py; ml/preprocessing.py; ml/model.py; ml/sequence_model.py; all three ml/models/*.pth checkpoints. MediaPipe background: https://arxiv.org/abs/2006.10214

# 2. Live decisions and training

### Live recognition

Static inference applies the checkpoint's preprocessing contract, softmax and a confidence threshold. Python maintains separate per-hand stabilization: normally five consecutive accepted frames confirm a sign, and ten missing predictions release it for repeat emission. Softmax confidence is a relative score among known labels; it is not a verified probability that an arbitrary gesture is a valid sign.

UnifiedSignRecognizer uses mean displacement of the wrist and five fingertips over an eight-frame history to route motion to the dynamic model and suppress static confirmation during movement. Displacement is per frame rather than per second. Dynamic predictions are stabilized over three windows. Its global gate watches hands[0], which can change identity and can miss movement confined to the other hand.

SequencePredictor stores the latest 30 processed frames. It starts predicting after 15 frames by padding earlier steps with zeros. Training capture/import defaults sample at 15 FPS and resample a complete gesture to 30 steps. Live inference instead adds each camera frame without timestamps or a 15 FPS sampling contract. At 30 FPS the window spans about one second; at 15 FPS it spans about two. The same physical sign therefore reaches the model at different temporal scales. A gesture that stops moving can also stop receiving dynamic decisions because the router only predicts while motion exceeds its threshold.

### Data preparation

Static webcam collection stores labeled normalized hand samples in CSV. Image extraction uses MediaPipe, label mapping, provenance groups, file deduplication, optional similarity filtering and an audit trail. The merger and GUIs support local collection workflows. Numerical validation and deduplication cannot verify sign correctness or recover missing signer identity.

Dynamic capture/import saves .npz sequences, labels, session identifiers and preprocessing metadata; quality gates reject insufficient detections or short clips. One accepted take gets a unique session identifier, which is useful for keeping its frames together but does not make it independent of other takes by the same signer.

### Training algorithm

Both trainers use seeded train/validation/test splits, inverse-frequency class weights, cross-entropy with 0.05 label smoothing, AdamW, weight decay 0.0001 and cosine learning-rate annealing. The recorded runs use 50 epochs and CPU training. Static training uses batch size 64 and initial learning rate 0.002; sequence training uses batch size 16 and rate 0.0015. The best checkpoint is selected by validation loss, then evaluated on test data.

Training-only augmentation applies small 3D rotations and coordinate jitter, with a 75% application probability. Sequence augmentation applies a consistent clip-wide rotation and a temporal interpolation routine described as speed warping. Inspect its effectiveness before relying on it: resampling the same full trajectory onto a shorter grid and back mostly changes interpolation smoothness rather than the gesture's duration within the window. Synthetic rotations of image-normalized coordinates also approximate viewpoint variation; they do not replace varied real captures.

Evidence: ml/train.py; ml/train_sequence.py; ml/dataset.py; ml/sequence_dataset.py; ml/video.py; ml/predict.py; ml/predict_sequence.py; ml/extract_dataset.py; ml/manual_recording.py; ml/dynamic_recording.py.

# 3. Data and evaluation findings

### Current static data

ml/data/landmarks.csv contains 159,987 rows, 24 labels and 1,639 session strings. Handedness counts are 98,909 Right and 61,078 Left. Label counts range from N=5,281 to L=8,252, a moderate 1.56x imbalance. More significant than overall imbalance is the uneven source distribution between splits.

The current CSV SHA-256 matches the canonical checkpoint and manifest: 17249bc657b7ed52f1d6a39f5d3bbbf6b8d2445efdab97f64094df68855647db. The legacy checkpoint instead references 695186db74e928760d81f63872f7bf89ec62e05c322bc93c7e2907fff2132ec6. It cannot be reevaluated honestly using this CSV as though it were its original training dataset.

| Model | Validation accuracy | Test accuracy | Test macro F1 |
|---|---|---|---|
| Canonical static, 24 labels | 97.64% | 53.19% | 59.49% |
| Legacy static, 26 labels | 98.57% | 48.03% | 61.87% |
| Dynamic J/Z | 100%, 2 clips | 100%, 2 clips | 100%, 2 clips |

Static figures above come from saved metrics; the canonical test was independently rerun for this review. The legacy result is historical because its matching CSV is not the current file. The models have different label sets and split allocations, so their numbers do not isolate the benefit of canonicalization.

### Canonical weaknesses

The lowest test F1 classes are P=18.56%, G=21.71%, N=22.81%, V=23.44% and D=32.88%. G recall is only 12.36% on 4,604 test examples. P recall is 90.85% but precision is only 10.34%, meaning many other signs are incorrectly predicted as P. Large test groups for G, I, K, M, O, Q and X dominate aggregate accuracy. W instead has 4,401 validation examples but only 274 test examples. Inspect confusion matrices and source groups before changing class weights or deleting difficult samples.

### What the evaluation protects

The split code keeps each session string within one partition, requires every label in each partition, rejects identical feature fingerprints across partitions, and checks dataset hashes. Evaluation reports accuracy, macro precision/recall/F1, weighted F1, per-class support and a confusion matrix. Macro F1 gives each class equal weight; accuracy and weighted F1 give larger classes more influence.

### What it does not establish

Imported identifiers such as ext_archive5_A_s0000 exist in the current CSV. The earlier review documents artificial grouping in older imports. These strings cannot establish independent people or recordings. New importer groups also incorporate labels, while trainers require a session to contain only one label. One person signing multiple labels can therefore cross partitions unless a separate global signer/source grouping is enforced. Exact-feature checks do not catch all near duplicates or shared-source dependence. Grouped evaluation should keep the actual person/source together across labels, consistent with scikit-learn's grouped-validation guidance: https://scikit-learn.org/stable/modules/cross_validation.html

The dynamic split records 7 train clips, 2 validation clips and 2 test clips, from 6 J and 5 Z examples. No .npz data were found in the searched project files outside packaged/build directories. Its data location, provenance and quality cannot be verified or its score reproduced here. Two successful test clips provide very weak evidence.

# 4. Bottlenecks and concrete problems

### Priority 1: static generalization and provenance

The 44.46 percentage-point canonical validation/test gap is the largest measured recognition problem. Source-domain shift, mislabeled data, viewpoint differences and preprocessing inconsistencies are plausible causes, not proven diagnoses. The test set has markedly different group sizes and class composition. Keep the difficult set as evidence while inspecting original images, handedness and label mappings by source. Do not improve the headline score by repeatedly tailoring the test set.

### Priority 1: missing dynamic evidence

A dynamic checkpoint now exists, but its 11-clip manifest is too small to support release claims and its source sequences are unavailable in this checkout. Recover and version the exact dataset before collecting broader claims or comparing backbones. Record signer, recording, camera/mirroring policy and source hashes separately from a take ID.

### Priority 1: temporal mismatch and missing-data handling

Live frame-count windows and per-frame displacement change behavior with camera speed. Missing frames are interpolated across gaps of any length; leading/trailing gaps hold the nearest observation. Original visibility is then lost from the classifier input. On failed normalization the predictor substitutes an all-zero window and still allows a model decision. A high confidence score could therefore reflect model bias rather than visible motion. Reject unusable windows explicitly and preserve quality masks/timestamps.

### Priority 2: routing and unknown gestures

The global gate follows the first detected hand rather than a stable pair or joint motion statistic. Threshold crossing can split a complete gesture, and dynamic training uses complete clips whereas live inference uses arbitrary sliding windows. J/Z are the only dynamic choices: unrelated motion can be classified as one of them. Neither existing closed-set score establishes false activations during rest, transitions, wrong signs or casual movements. Thresholds require calibration on representative negative examples.

### Priority 2: browser completion and parity

web_client/asignx_client_sdk.ts contains normalization, coaching and stabilization helpers, but no ONNX Runtime session or camera application was found there. Its static normalization now supports the canonical metadata contract, resolving part of the older report's concern. Its motion gate still uses only instantaneous wrist displacement, while Python uses wrist plus fingertips and a history. The TypeScript stableLabel can remain stale on low-confidence frames; missingStreak is reset on every update, so release behavior differs from Python when the hand stays visible but predictions are rejected.

ml/export_onnx.py catches any ONNX export exception and substitutes TorchScript, then the CLI describes the bundle as exported. A TorchScript fallback is not a usable ONNX Runtime browser graph. No .onnx artifact was found in searched project files outside packaged/build directories. Make required export failure explicit and verify logits through the intended runtime before describing browser support as complete.

### Priority 2: scope and measurement

Hand-only inputs cannot distinguish signs whose meaning requires body-relative location or facial/nonmanual signals. Keep the current alphabet/selected-gesture scope explicit. No camera-to-decision latency, detector recall on a labeled corpus, sustained device FPS or continuous-stream false-activation benchmark was run here. It would be speculation to call model compute the main speed bottleneck; profile capture, detection, inference and rendering separately first.

Evidence: current metrics and manifests; ml/predict.py; ml/predict_sequence.py; ml/preprocessing.py; ml/export_onnx.py; web_client/asignx_client_sdk.ts; docs/ai-state-review.md for historical provenance concerns.

# 5. Next steps within project scope

### 1. Freeze an auditable baseline

Keep the canonical static model plus J/Z as the current intended vocabulary. Archive checkpoint, metrics, split manifest, CSV hash and working-tree evidence together. Recover the dynamic sequences or explicitly mark that experiment unreproducible. Unify launch defaults: README's combined test uses canonical static, while ml.predict still defaults to the legacy model. Deliverable: one build manifest that identifies the actual models launched.

### 2. Diagnose static errors before retraining

Review original samples for G/I/K/M/O/Q/X and the most frequent confusion pairs, separated by source and handedness. Verify labels, detector coordinates, mirror policy and preprocessing once per source. Preserve corrections as a versioned change log. Do not fabricate session IDs or silently relabel ambiguous images. Deliverable: source-by-class error table and a reviewed correction set.

### 3. Establish global groups and a fresh benchmark

Add signer_id, recording_id, source_id and device metadata. Keep a signer/source entirely within one split across all labels. Freeze a new-signer test set, plus a separate realistic negative/idle stream set. Use grouped validation within the remaining data to choose settings. Existing test results are already part of development knowledge; a fresh untouched final benchmark is needed after substantial diagnosis. Deliverable: a benchmark manifest with provenance and per-sign support, plus detector failure/coverage counts.

### 4. Repair live temporal contracts

Sample live sequences using timestamps at the training rate, or explicitly store and train against the intended temporal contract. Express velocity and hold/release periods in time units. Use stable hand identities and both-hand motion evidence. Bound interpolation gaps, preserve validity masks, reject insufficient visibility, and evaluate completed-gesture boundaries and repeated-sign behavior. Deliverable: recorded-stream replay checks at multiple frame rates, occlusions and camera stalls.

### 5. Collect realistic J/Z and negative examples

Collect across people, days, speeds, handedness and devices using the existing Dynamic Studio. As a collection starting target, use at least 10 signers with several independent takes per class; this is a planning target, not a statistically guaranteed sample size. Include idle hands, transitions, partial/wrong attempts and unrelated motion. Reserve people before training. Compare the available LSTM and TCN-attention on identical groups. Deliverable: reproducible per-class clip metrics plus false activations per minute and gesture-to-emission delay on continuous streams.

### 6. Complete a narrow browser integration

After the recognition contracts are stable, require a real ONNX graph, verify Python/TypeScript normalization and output logits on fixed examples, and align threshold/release behavior. Integrate camera/model loading and measure supported devices. MediaPipe's web detection calls are synchronous; worker isolation is an available method to avoid blocking the page:
https://developers.google.cn/edge/mediapipe/solutions/vision/hand_landmarker/web_js

### 7. Set measurable release gates

Choose numerical targets before final evaluation: minimum per-class recall/precision, acceptable false emissions per minute, repeat-sign success, p50/p95 delay and detector coverage. Report metrics by signer, source, handedness and device alongside overall accuracy. Expand vocabulary only after current labels pass those gates. Body/face inputs should be introduced only if the agreed next sign inventory requires them; sentence translation and a large language model are outside this immediate scope.

# 6. Verification and repeatable build updates

### What was checked for this review

All 100 repository tests passed under Python 3.11, with 13 tracing/deprecation warnings. Checkpoint metadata and parameter counts were inspected directly. The current CSV was counted and hashed, and model/split identity was checked. The canonical test was rerun without retraining; its output is retained at tmp/canonical-review-test.json. No webcam trial, raw-image label audit, dynamic-data reevaluation or browser/mobile benchmark was performed. Passing tests should not be presented as an end-user accuracy result.

### Build identity

Reviewed date: 19 October 2026, Asia/Singapore. Git HEAD at review: 9050c234d3e346223045e64f3a2e13501d65ccac. The checkout contains extensive uncommitted changes, so that commit alone does not reproduce the reviewed system. The report builder records SHA-256 hashes of relevant current source, model, manifest and metrics files in a companion evidence JSON. Such hashes identify inspected bytes; they do not replace archiving those files.

### Update this report every build

The editable narrative is docs/system-state-current.md. Rebuild with py -3.11 docs/build_system_state_report.py. The builder refreshes the evidence JSON and PDF at output/pdf/system-state-current.pdf; it does not automatically rewrite interpretations, run tests or evaluate models. Review and update this narrative first so stale prose is not mistaken for a fresh audit.

For each new build: archive the previous PDF/source/evidence; rerun tests; reevaluate compatible checkpoints against their matching datasets/manifests; replace the numerical findings and verification dates in the source; review changes to preprocessing, labels, groups and deployment; rebuild and inspect rendered pages.

### Static reproduction command

```
py -3.11 -m ml.evaluate `
  --model ml/models/sign_classifier_canonical.pth `
  --output tmp/canonical-review-test.json
```

### Repository test command

```
py -3.11 -m pytest -q
```

### Report dependencies

Python 3.11 with reportlab, pymupdf and Pillow. Install these explicitly for a clean environment. Rendered page PNGs/contact sheets go to tmp/pdfs/system-state/.

### Primary repository references

asignx/detector.py and main.py: tracking and launch wiring. ml/model.py and ml/sequence_model.py: model definitions. ml/preprocessing.py and ml/video.py: normalization, augmentation and sequence preparation. ml/dataset.py and ml/sequence_dataset.py: grouping and integrity checks. ml/train.py, ml/train_sequence.py and ml/evaluate.py: optimization and scoring. ml/predict.py and ml/predict_sequence.py: live decisions. ml/models/: inspected checkpoints, metrics and manifests. ml/data/landmarks.csv: current static dataset. ml/export_onnx.py and web_client/asignx_client_sdk.ts: browser export/helpers.

### External sources consulted on 19 October 2026

MediaPipe Hands paper: https://arxiv.org/abs/2006.10214 . scikit-learn grouped evaluation guide: https://scikit-learn.org/stable/modules/cross_validation.html . MediaPipe web hand-landmarker guide: https://developers.google.cn/edge/mediapipe/solutions/vision/hand_landmarker/web_js . Recommendations are project-specific judgments, not measured guarantees.